Entrega 6 – Importación de datos externos

TP Mundial de Fútbol 2026

Bases de Datos Aplicada

Comisión 5600

Grupo 05

Vázquez Nahuel Domiciano

González Esteban Gabriel

7/11/2026

Introducción

La base se completa con dos datasets en formatos distintos (CSV y XML) y cuatro fuentes web consultadas desde T-SQL. Todo se importa con stored procedures, sin modificar los archivos de origen y con importación parcial: un registro inválido se informa y no frena al resto.

Fuente	Tipo	Formato	Qué carga	Procedimiento
Estadios históricos (jfjelstul/worldcup)	Dataset descargado	CSV	Sedes y ciudades	`Torneo.Importar_CSV_Stadiums`
Wikipedia, Mundial 2026	Dataset por scraping	XML	Sedes, ciudades, fases, grupos, selecciones y partidos con resultados	`Torneo.Importar_XML_Mundial`
mledoze/countries	JSON por HTTP	JSON	Países, monedas, nombre en inglés, código FIFA y coordenadas	`Participantes.Importar_API_Paises`
Nominatim (OpenStreetMap)	API	JSON	Latitud y longitud de las sedes	`Torneo.Actualizar_Coordenadas_Sedes`
Banco Mundial	API	JSON	PBI per cápita de cada país	`Participantes.Importar_API_PBI`

Fuente	Tipo	Formato	Qué carga	Procedimiento
timeapi.io	API	JSON	Huso horario de sedes y países	`Torneo.Actualizar_Husos_Horarios`

Los scripts son `11_SP_Importaciones.sql` y `13_SP_Importaciones_XML_APIs.sql`; las pruebas, `12` y `14`. El scraper está en `scraping/scraper_mundial_2026.py`.

Esquema general de importación

Todas las importaciones siguen el mismo circuito: leer la fuente, validar cada registro, guardar los válidos y dejar constancia en el log.

1. **Lectura.** Los archivos se leen desde la carpeta del servidor (`C:\Importar\`): el CSV con `BULK INSERT` sobre una vista de staging y el XML con `OPENROWSET(BULK …, SINGLE_BLOB)`, con SQL dinámico porque la ruta no puede ir en una variable. Las fuentes web se consultan con `Torneo.Consultar_API`.
2. **Staging.** Cada registro queda en una tabla de staging (`Torneo.Staging_stadiums`, `Torneo.Staging_fixture`) o en una tabla temporal, en estado `Pendiente`.
3. **Validación por registro.** Se acumulan todos los problemas de cada fila en `mensaje_error` (datos faltantes, longitudes, país o estadio inexistente, números inválidos). Los repetidos dentro del mismo archivo se detectan con `ROW_NUMBER()` y quedan `Duplicado`.
4. **Upsert.** Las filas válidas se insertan si no existen o se actualizan si ya estaban (`Insertado` / `Actualizado`), dentro de una transacción con `TRY/CATCH` y `ROLLBACK`. Reimportar el mismo archivo no genera duplicados.
5. **Log.** Cada ejecución agrega una fila en `Torneo.Log_importacion` con la fuente, la fecha, los registros procesados y con error, y el estado (`Completado`, `Completado con errores`, `Sin registros válidos` o `Fallido`).

Los errores que impiden toda la importación (ruta vacía, archivo inexistente, XML mal formado, respuesta inválida de una API) se informan con `THROW`. Los recorridos fila por fila contra las APIs usan un `WHILE` sobre una tabla numerada, sin cursores.

Consumo de APIs. `Torneo.Consultar_API` hace un GET con el objeto `MSXML2.ServerXMLHTTP.6.0` mediante `sp_OACreate`, por lo que requiere habilitar `Ole Automation Procedures` (lo hace el script 11). La respuesta se lee con `INSERT … EXEC sp_OAGetProperty` para admitir textos de más de 4000 caracteres, y se procesa con `OPENJSON` y `JSON_VALUE`. Valida la URL vacía, la creación del objeto, la conexión y que el estado HTTP sea 200.

Dataset 1: estadios históricos (CSV)

El archivo `stadiums.csv` del repositorio jfjelstul/worldcup trae los 240 estadios usados en todos los mundiales y se importa sin modificar.

Aspecto	Detalle
Origen	`data-csv/stadiums.csv`, copiado tal cual en `datos/stadiums.csv`
Columnas	key_id, stadium_id, stadium_name, city_name, country_name, stadium_capacity, stadium_wikipedia_link, city_wikipedia_link
Lectura	`BULK INSERT` con `FORMAT = 'CSV'` y `CODEPAGE = '65001'` (UTF-8, respeta comillas y tildes) sobre la vista `Torneo.vw_Staging_stadiums`
Procedimientos	`Torneo.Importar_CSV_Stadiums` lee el archivo y llama a `Torneo.Procesar_CSV_Stadiums`, que valida y hace el upsert en `Torneo.Sede`
Clave del upsert	Nombre del estadio + ciudad
Vinculación	El país se busca por nombre en castellano, nombre en inglés o código ISO; si la ciudad no existe, se crea
Validaciones	Estadio, ciudad o país faltantes; país inexistente; estadio de más de 30 caracteres o ciudad de más de 50; capacidad no numérica o no positiva; filas repetidas en el archivo

Las coordenadas y el huso horario no vienen en el CSV: quedan en NULL y los completan después Nominatim y timeapi.io.

En la prueba completa (script 14, con los países ya importados), la carga dio 240 filas leídas, 227 insertadas, 13 actualizadas y ninguna con error. El archivo `datos/stadiums_prueba_errores.csv` tiene 7 filas con errores a propósito para probar cada validación.

Dataset 2: scraping de Wikipedia (XML)

El fixture completo del Mundial 2026 (104 partidos, 12 grupos y 16 sedes, con resultados, asistencia y árbitro) se obtiene por scraping de Wikipedia y se guarda como XML.

Páginas. 2026 FIFA World Cup Group A a Group L, 2026 FIFA World Cup knockout stage, 2026 FIFA World Cup final y el artículo de cada estadio (para la capacidad).

Técnica. El script `scraping/scraper_mundial_2026.py` (Python, solo biblioteca estándar) descarga el código fuente de cada página (`?action=raw`) en lugar del HTML, porque es más estable. Después:

- Busca los bloques de partido (`{{#invoke:football box|main …}}` o `{{Football box …}}`) con un parser de llaves balanceadas y separa sus parámetros.
- Toma las selecciones del código FIFA de la plantilla de bandera (`{{#invoke:flag|fb-rt|MEX}}`), con el título del partido como respaldo.
- Convierte la hora de 12 a 24 horas (`1:00 p.m.` → 13:00) y calcula la hora UTC con el desfase que indica la página.
- Deduce la fase por el título de la sección (Round of 32 → Dieciseisavos de final, etc.) y sigue las páginas vinculadas de la eliminatoria y la final, sin repetir partidos.
- Hace una pausa de 1 segundo entre pedidos y se identifica con un User-Agent, como pide Wikipedia.

Se ejecuta con `python scraper_mundial_2026.py` y genera `mundial_2026_wikipedia.xml` :

```
<mundial fuente="…" fecha_extraccion="…">
  <sedes><sede estadio="Estadio Azteca" ciudad="Mexico City" pais_iso="MEX"
capacidad="87523" huso_horario="UTC-6"/></sedes>
  <grupos><grupo letra="A"><seleccion codigo_fifa="MEX" nombre="Mexico"/>
</grupo></grupos>
  <partidos>
    <partido numero="1" fase="Fase de Grupos" grupo="A">
      <local codigo_fifa="MEX">Mexico</local><visitante
codigo_fifa="RSA">South Africa</visitante>
      <fecha_hora_local>2026-06-11 13:00:00</fecha_hora_local>
<fecha_hora_utc>2026-06-11 19:00:00</fecha_hora_utc>
      <estadio>Estadio Azteca</estadio><goles_local>2</goles_local>
<goles_visitante>0</goles_visitante>
      <asistencia>80824</asistencia><arbitro>Wilton Sampaio (Brazil)
</arbitro>
    </partido>
  </partidos>
</mundial>
```

Importación. `Torneo.Importar_XML_Mundial` lee el archivo sin modificarlo y lo recorre con `.nodes()` y `.value()` :

1. Inserta las fases y grupos que falten.

2. Hace el upsert de sedes y ciudades. Guarda como capacidad la mayor entre la de la ficha y la asistencia máxima, porque algunos estadios se reconfiguran para el Mundial.
3. Hace el upsert de selecciones por país (código FIFA o ISO) con su grupo.
4. Carga los partidos en `Torneo.Staging_fixture` y llama a `Torneo.Importar_Fixture` (Entrega 5), que valida cada partido y hace el upsert en `Torneo.Partido`.

En la prueba se importaron los 104 partidos sin errores; reimportar el archivo actualiza los 104 sin duplicar. El archivo `datos/mundial_2026_prueba_errores.xml` prueba las validaciones.

APIs y fuentes web

Tres APIs públicas y un JSON por HTTP completan lo que los datasets no traen: países, coordenadas, poder adquisitivo y husos horarios. Ninguna requiere clave.

Fuente	URL	Datos que se usan	Límites y requisitos	Procedimiento
mledoze/countries	`https://raw.githubusercontent.com/mledoze/countries/master/dist/countries.json`	Nombre en castellano e inglés, código ISO alfa-3, código COI (como código FIFA), moneda y coordenadas de referencia	Archivo de unos 800 KB, sin límite publicado	`Participantes.Importar_API_Paises`
Nominatim (OpenStreetMap)	`https://nominatim.openstreetmap.org/search`	Latitud y longitud de cada estadio	1 pedido por segundo por IP; exige un User-Agent que identifique la aplicación	`Torneo.Actualizar_Coordenadas_Sedes`
Banco Mundial	`https://api.worldbank.org/v2/country/all/indicator/NY.GDP.PCAP.CD`	PBI per cápita en USD corrientes, último `s N años publicados (mrnev=N), con historial por año`	Hasta 32.500 registros por página	`Participantes.Importar_API_PBI`
timeapi.io	`https://timeapi.io/api/timezone/coordinate`	Desfase estándar y si se usa horario de verano en unas coordenadas	Sin límite publicado; se hace 1 consulta por segundo	`Torneo.Actualizar_Husos_Horarios`

Países. Se hace un upsert por código ISO. A los países existentes solo se les completan el nombre en inglés, el código FIFA y las coordenadas; los nuevos se insertan con su moneda, que se crea si no existe. El código FIFA se toma del código olímpico (GER, NED, SUI, RSA), con una tabla de excepciones donde la FIFA usa otro (IRN, IDN, SVN, KVX, CUW, entre otros). En la prueba se recibieron 250 países: 246 cargados y 4 territorios sin moneda rechazados.

Se usa este JSON porque la API REST Countries, prevista al principio, dio de baja sus versiones 1 a 4 en 2026: la v3.1 ahora responde con un aviso de versión discontinuada en lugar de los datos, y la v5 exige registrarse y una clave. mledoze/countries es el conjunto de datos sobre el que estaba construida REST Countries, con los mismos campos.

PBI. Es información histórica, así que se conserva el registro de cada año: se traen los últimos N valores de cada país (@cantidad_anios, 5 por defecto) y se hace un upsert en Participantes.Historial_pbi_pais con clave país + año. Reimportar agrega los años nuevos y actualiza los revisados por el Banco Mundial, sin borrar los anteriores; Pais.pbi_per_capita queda con el año más reciente. Los agregados regionales (por ejemplo "América Latina y el Caribe") quedan `Omitido` y los países sin valor quedan `Sin dato`. Se usa como indicador de poder adquisitivo para asignar publicidad. En la primera prueba (un solo año) se actualizaron 211 países.

Coordenadas. Para cada sede sin coordenadas se busca "estadio, ciudad" en Nominatim, con pausa de 1 segundo. Si no hay resultado se informa y la sede queda pendiente para un nuevo intento. En la prueba se geolocalizaron 18 de 20 sedes; las 2 restantes son estadios que ya no existen o cambiaron de nombre.

Husos horarios. Con las coordenadas de cada sede y de cada país se consulta timeapi.io. Como el torneo se juega en junio y julio, en el hemisferio norte se suma 1 hora si el lugar usa horario de verano. El resultado se guarda como texto (`UTC-3` , `UTC+5:30`). En la prueba se completaron 48 husos sin errores.

Ejecución y pruebas

La importación completa se corre en este orden, y cada paso usa lo que cargó el anterior:

1. `Participantes.Importar_API_Paises` : países y monedas (los demás procesos buscan el país por nombre o código).
2. `Participantes.Importar_API_PBI` : PBI per cápita.
3. Alta manual de Inglaterra y Escocia con `Participantes.Alta_Pais` , porque juegan el Mundial pero no son países en ISO 3166.
4. `Torneo.Importar_XML_Mundial @nombre_archivo = 'C:\Importar\mundial_2026_wikipedia.xml'` : fixture, sedes y selecciones.
5. `Torneo.Importar_CSV_Stadiums @nombre_archivo = 'C:\Importar\stadiums.csv'` : estadios históricos.
6. `Torneo.Actualizar_Coordenadas_Sedes` : coordenadas de las sedes.
7. `Torneo.Actualizar_Husos_Horarios` : husos de sedes y países (necesita las coordenadas del paso 6).

Requisitos.

- Permisos de sysadmin para habilitar `Ole Automation Procedures` (una sola vez, lo hace el script 11).
- Conexión a Internet en el servidor de SQL Server.

- Carpeta `C:\Importar\` en la PC del servidor, con permiso de lectura para el usuario del servicio, con los archivos `stadiums.csv`, `stadiums_prueba_errores.csv`, `mundial_2026_prueba_errores.xml` y `mundial_2026_wikipedia.xml`.
- SQL Server 2019 o posterior (la función `Torneo.fn_Url_Encode` usa la intercalación UTF-8).
- Python 3 solo para volver a generar el XML; el archivo generado ya está en el repositorio.

Pruebas. `12_test_SP_Importaciones.sql` (10 casos) y `14_test_SP_Importaciones_XML_APIs.sql` cubren la importación real, la reimportación sin duplicados, los archivos con errores a propósito, los archivos inexistentes, las rutas o URLs vacías, una URL de dominio inválido y el contenido del log. Cada caso indica arriba su "Resultado esperado"; los errores en rojo de los casos inválidos son el resultado correcto.

Limitaciones y decisiones

- **Wikipedia puede cambiar su formato.** El scraper depende de las plantillas de partido y de bandera. Para diagnosticar cambios, guarda lo descargado en `scraping/wikitext_crudo/` (excluida del repositorio) y avisa si algún partido queda sin selecciones.
- **Coordenadas de países aproximadas.** mledoze/countries da el centro geográfico del país, no su capital. En países con varios husos, el huso del país puede no coincidir con el de la capital (Estados Unidos queda en UTC-5). Las sedes sí usan las coordenadas exactas del estadio.
- **Horario de verano simplificado.** Se suma 1 hora en el hemisferio norte si el lugar usa horario de verano, porque el torneo es en junio y julio.
- **Nominatim no encuentra estadios demolidos o renombrados** (Chateau Carreras, Pocitos, White City): quedan sin coordenadas y se informan.
- **El PBI se usa con su último año.** Se conserva el historial por año en Historial_pbi_pais; la publicidad usa solo el valor del año más reciente.
- **Selecciones nuevas sin confederación.** El XML no la trae; se completa con `Participantes.Modificacion_Seleccion`.
- **El árbitro del XML no se importa.** Viene como texto (nombre y país) y la designación arbitral necesita categoría y rol, que se cargan por otros medios.

Tablas cargadas por otros medios. Como permite la consigna, las tablas que no se completan por importación o API se cargan con los SP de alta o con el script de datos de prueba:

Origen	Tablas
Importación y APIs	Pais, Historial_pbi_pais, Moneda, Ciudad, Sede, Fase, Grupo_torneo, Seleccion, Partido, Log_importacion (y las tablas de staging)
Catálogos con SP de alta	Confederacion, Posicion, Categoria_arbitro, Rol_arbitro, Idioma, Esquema_tactico, Tipo_gol, Periodo_partido, Tipo_tarjeta, Motivo_tarjeta, Franja_horaria_tarifa, Parametros_suspension
Datos de prueba (SP de alta)	Cotizacion_moneda, Feriado_pais, Club_origen, Arbitro, Idioma_arbitro, Jugador, Integrante_cuerpo_tecnico, Convocatoria, Novedad_convocatoria, Anunciante, Campania_publicitaria, Mercado_interes_campania, Pieza_publicitaria
Operación del torneo (SP de lógica de negocio)	Designacion_arbitral, Alineacion_partido, Detalle_alineacion_jugador, Gol, Tarjeta, Suspension_jugador, Sustitucion, Informe_post_partido, Exhibicion_publicidad_partido